

Bug Report: `react-native-bare-kit` native library links against private Android system library, breaking module registration on Android

Summary

`libbare-kit.so`, the core native library shipped inside `react-native-bare-kit@0.15.0` (a transitive dependency of `@qvac/sdk@0.19.0` via its Android autolinking), has a direct ELF dependency (`DT_NEED`) on `libnativehelper.so` — a private Android platform library that apps are not permitted to `dlopen` on modern Android. This causes native module loading to fail on real Android devices, in two different ways depending on whether React Native's New Architecture is enabled, and in neither case does `bare-kit` actually work.

We believe this is a packaging defect in the native `.so` shipped for the Android target of `react-native-bare-kit` (and/or `@qvac/sdk`'s bundling of it), not something fixable from the consuming app's configuration — we tested every combination of `targetSdkVersion` and New Architecture on/off available to us and could not get `bare-kit` to initialize successfully on a physical Android device.

Environment

- `@qvac/sdk`: `0.19.0`
- `react-native-bare-kit`: `0.15.0` (latest published version at time of testing)
- `qvac.config.json` plugins: `["@qvac/sdk/llamacpp-completion/plugin"]`
- Expo plugins in `app.json`: `expo-build-properties`, `@qvac/sdk/expo-plugin`
- Build config: `compileSdk 36`, `buildTools 36.0.0`, `minSdk 29`, NDK `29.0.14206865`, Kotlin `2.1.20`, KSP `2.1.20-2.0.1`, Hermes enabled
- `targetSdkVersion` tested: `36` (default) and `31` (minimum allowed by `expo-build-properties` in this Expo SDK version)
- Test device: physical Android phone, Samsung SM-S938B (not an emulator — we're aware QVAC currently doesn't support emulators)
- Architecture: `arm64-v8a` only

Root cause

1. Initial symptom

On app launch, before any QVAC/bare-kit API is called, the app crashes with:

```
Invariant Violation: TurboModuleRegistry.getEnforcing(...): 'PlatformConstants' could not be found.  
Verify that a module by this name is registered in the native binary.
```

This is misleading — it has nothing to do with `PlatformConstants` itself. We confirmed this by building a bare vanilla Expo app with no QVAC/bare-kit dependency at all, which launched cleanly. The crash only occurs once `@qvac/sdk` (and therefore `react-native-bare-kit`) is added.

2. What's actually failing (from logcat)

The device logcat (properly decoded — it's emitted as UTF-16LE) shows the real failure happening earlier, during native library loading:

```
SoLoader: Running a recovery step for libappmodules.so due to  
com.facebook.soloaders.SoloaderDSONotFoundError: couldn't find DSO to load: libnativehelper.so  
...  
SoLoader: Failed to recover
```

On New Architecture, React Native merges all TurboModule provider native code into a single `libappmodules.so`. Because that merged library transitively depends on `libbare-kit.so`, and `libbare-kit.so` cannot load, **the entire merged native module registry fails to load** — not just the `BareKit` module. This is why the very first TurboModule lookup React Native itself performs at startup (`PlatformConstants`) throws, even though it has no relation to QVAC.

3. Confirming the exact culprit

We extracted the built debug APK and ran `readelf -d` against every `.so` file packaged for `arm64-v8a` (48 libraries total, including the full HoleyPunch "Bare" ecosystem and seven `libqvac-ggml-cpu-*` variants for different ARM ISA levels). Only one library declares `libnativehelper.so` as a direct dependency:

```
== libbare-kit.so ==  
NEEDED    libandroid.so  
NEEDED    libnativehelper.so    <-- private Android platform library  
NEEDED    libdl.so  
NEEDED    liblog.so  
NEEDED    libm.so  
NEEDED    libc++_shared.so  
NEEDED    libc.so
```

And `libappmodules.so` (the New Architecture merged module registry) directly requires `libbare-kit.so`:

```

== libappmodules.so ==
  NEEDED    libfbjni.so
  NEEDED    libjsi.so
  NEEDED    libreactnative.so
  NEEDED    libbare-kit.so    <-- pulls in the broken dependency
  NEEDED    libm.so
  NEEDED    libc++_shared.so
  NEEDED    libdl.so
  NEEDED    libc.so

```

`libnativehelper.so` is a private/internal Android platform library. Apps are not allowed to `dlopen` it directly on modern Android — this is enforced via the OS's native library visibility/linker-namespace restrictions, and appears to be additionally gated by the app's declared `targetSdkVersion`.

4. `targetSdkVersion` experiment

Lowering `targetSdkVersion` from `36` to `31` (the lowest value `expo-build-properties` allows in our Expo SDK version) did unblock loading `libnativehelper.so`, and the total crash on `PlatformConstants` went away. This confirms the restriction is tied at least partly to the consuming app's declared target API level, not purely the OS version of the device.

5. The failure persists regardless of architecture mode

With `targetSdkVersion 31` fixed, we tested both React Native architecture modes:

- **New Architecture ON** (`newArchEnabled: true`): the exact same total failure returns (`PlatformConstants could not be found`), because enabling it re-merges `libbare-kit.so` into `libappmodules.so`, reintroducing the broken dependency chain into the single combined library.
- **New Architecture OFF** (`newArchEnabled: false`): the total failure disappears (other native modules load fine), but a new, `BareKit`-specific failure appears when the module is first accessed:

```

Exception in HostObject::get for prop 'BareKit': java.lang.NullPointerException: get
(...) must not be null
    at com.facebook.react.bridge.ModuleHolder.create(ModuleHolder.kt:168)
    at com.facebook.react.bridge.ModuleHolder.getModule(ModuleHolder.kt:132)
    ...

```

This happens because `bare-kit`'s Android module appears to be implemented purely as a TurboModule/JSI HostObject and does not have a working code path under the legacy bridge, so it throws when the legacy bridge's `ModuleHolder` tries to instantiate it. Downstream, `@qvac/sdk` itself then throws `TypeError: Cannot read property 'prototype' of undefined` when it tries to use the (unavailable) `BareKit` binding.

We tried every combination available to us (`targetSdkVersion` 31/36 × New Architecture on/off) and could not find one where `bare-kit` actually initializes successfully on this physical device.

Reproduction steps

1. Create an Expo app and add `@qvac/sdk@0.19.0` (pulls in `react-native-bare-kit@0.15.0` transitively) with `qvac.config.json` containing `{"plugins": ["@qvac/sdk/llamacpp-completion/plugin"]}`.
2. `npx expo prebuild --clean && npx expo run:android` on a physical Android device (arm64-v8a).
3. Observe the crash on launch (New Architecture on) or on first `BareKit` access (New Architecture off).
4. To confirm the dependency issue directly, without needing to reproduce the crash: unzip the built APK, and for any `.so` under `lib/arm64-v8a/`, run:

```
readelf -d lib/arm64-v8a/libbare-kit.so | grep NEEDED
```

`libnativehelper.so` appears in the output.

What we've already tried (all unsuccessful at making `bare-kit` fully functional)

- Confirmed the issue is not our toolchain/environment by reproducing a clean launch with a vanilla Expo app containing no QVAC dependency.
- Updated to the latest published `react-native-bare-kit` (`0.15.0`) — already on it, issue persists.
- Lowered `targetSdkVersion` to `31` (minimum allowed) — fixes the total crash, but only by disabling New Architecture, which then breaks `bare-kit` specifically instead.
- Toggled `newArchEnabled` both ways in combination with the above — every combination fails in one of the two ways described above.

What would help

- A rebuilt Android native binary for `react-native-bare-kit` that does not link against `libnativehelper.so` (or statically links/replaces whatever functionality it's using from it), and/or a `BareKit` module implementation that also works correctly under the legacy bridge (or explicit confirmation that New Architecture is required, with guidance on why the merged-library path currently fails).
- Confirmation of whether this is a known issue, and whether other teams building with `@qvac/sdk` on Android are hitting the same failure.

Current workaround in our app (for context, not a fix)

We deferred the `@qvac/sdk` import to load lazily on user interaction (instead of at module/app load time) and kept our existing `try/catch` around it, plus set `newArchEnabled: false` and `targetSdkVersion: 31`. This lets the rest of the app run normally and surfaces the `BareKit` failure as a catchable JS error instead of crashing the app on launch — but the on-device LLM feature itself remains non-functional on Android.